

贪心算法

《算法设计与分析》

信息科学与技术学院 曾华琳



03



0 / 1 背包问题



4.2 贪心算法的基本要素

- 0-1背包问题:

给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

在选择装入背包的物品时，对每种物品 i 只有2种选择，即装入背包或不装入背包。不能将物品 i 装入背包多次，也不能只装入部分的物品 i 。



4.2 贪心算法的基本要素

- 背包问题:

与0-1背包问题类似，所不同的是在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包， $1 \leq i \leq n$ 。

这2类问题都具有最优子结构性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。



4.2 贪心算法的基本要素

例

●例：若 $n = 3$, $w = (10, 20, 30)$, $v = (60, 100, 120)$, $c = 50$, 则

➤对于0-1背包问题，可行解为：

$$(x_1, x_2, x_3) = (0, 1, 1) = 220$$

$$(x_1, x_2, x_3) = (1, 1, 0) = 160$$

$$(x_1, x_2, x_3) = (1, 0, 1) = 180$$

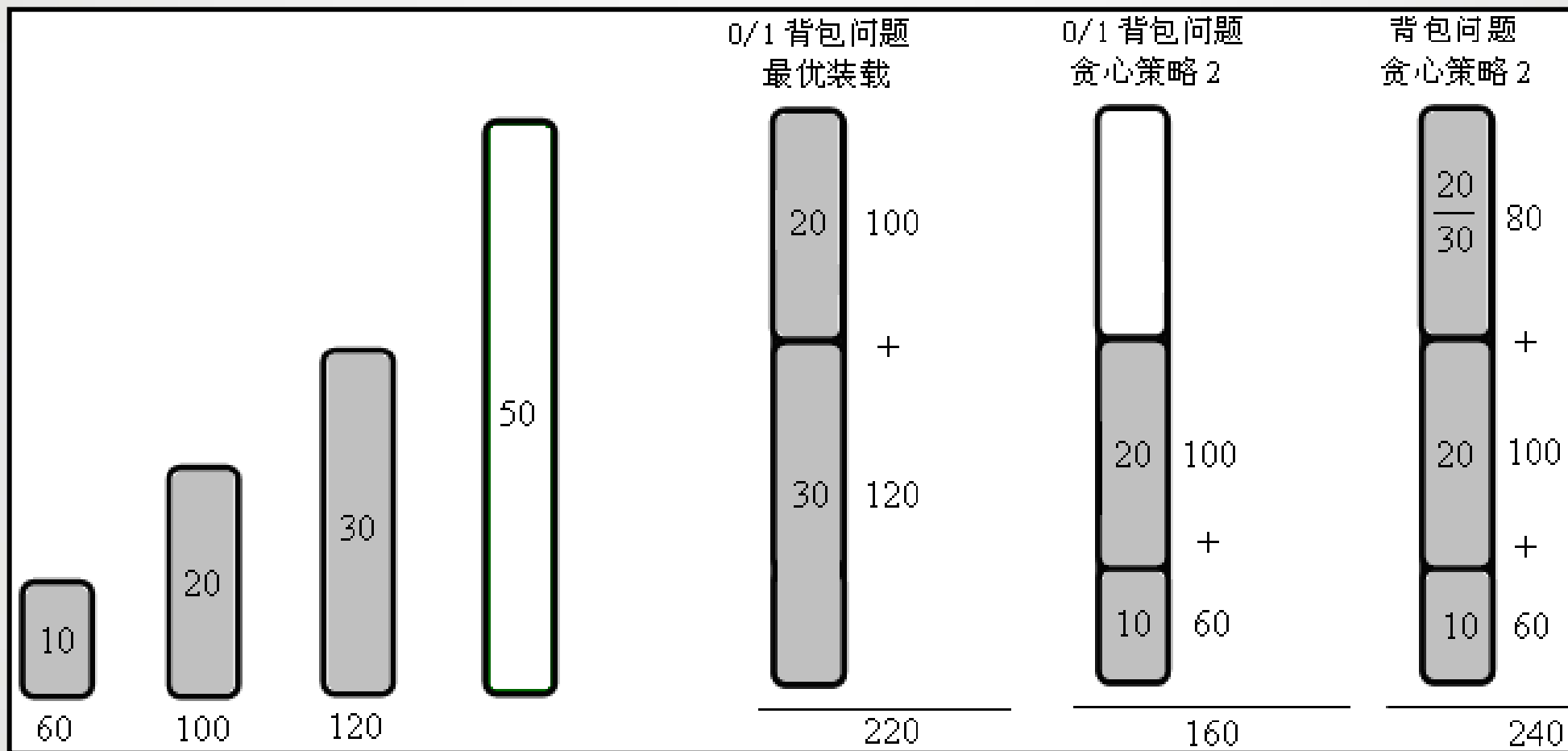
最优解为：选择物品2和物品3，总价值为220

➤对于小数背包问题，按照物品价值率最大的贪心选择策略，其解为(10, 20, 20)，总价值为240。



背包问题

■ 例子





4.2 贪心算法的基本要素

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值 V_i/W_i ，然后，依贪心选择策略，将尽可能多的单位重量价值最高的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过 C ，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。



4.2 贪心算法的基本要素

对于**0-1 背包问题**，贪心选择之所以不能得到最优解是因为在这种情况下，它无法保证最终能将背包装满，部分闲置的背包空间使每公斤背包空间的价值降低了。事实上，在考虑 0-1 背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。由此就导出许多互相重叠的子问题。这正是该问题可用**动态规划算法**求解的另一重要特征。

实际上也是如此，动态规划算法的确可以有效地解0-1背包问题。



4.2 贪心算法的基本要素

具体算法可描述如下页：

```
void Knapsack(int n,float M,float v[],float w[],float x[])
{
    Sort(n,v,w);
    int i;
    for (i=1;i<=n;i++) x[i]=0;
    float c=M;
    for (i=1;i<=n;i++) {
        if (w[i]>c) break;
        x[i]=1;
        c-=w[i];
    }
    if (i<=n) x[i]=c/w[i];
}
```

算法knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此，算法的计算时间上界为 $O(n\log n)$ 。

为了证明算法的正确性，还必须证明背包问题具有贪心选择性质。

- **问题描述**: 给定n种物品和一个背包。物品i的重量是 W_i ，其价值为 V_i 。背包的容量为C。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？这里，在选择物品i装入背包时，**可以选择物品i的一部分**，而不一定要全部装入背包。

$$\begin{aligned} \max & \sum_{i=1}^n v_i x_i && x_i \text{ 为装入物品 } i \text{ 的比例} \\ \begin{cases} \sum_{i=1}^n w_i x_i \leq c & w_i > 0 \\ 0 \leq x_i \leq 1 & i = 1, 2, \dots, n \end{cases} && \begin{aligned} & w_i \text{ 为重量, } c \text{ 为背包容量} \\ & v_i \text{ 为价值} \\ & v_i/w_i \text{ 为价值率(单位重量价值)} \end{aligned} \\ v_i > 0, w_i > 0, c > 0 & i = 1, 2, \dots, n \end{aligned}$$

例子: $n=3, c=20, v=(25, 24, 15), w=(18, 15, 10)$ ，列举4个可行解：

	(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum v_i x_i$
①	$(1/2, 1/3, \frac{1}{4})$	16.5	24.5
②	$(1, 2/15, 0)$	20	28.2
③	$(0, 2/3, 1)$	20	31
④	$(0, 1, \frac{1}{2})$	20	31.5 (最优解)

贪心策略设计:

- 策略1: 按价值最大贪心, 是目标函数增长最快。
按价值排序从高到低选择物品→②解(次最优)
- 策略2: 按重量最小贪心, 使背包增长最慢。
按重量排序从小到大选择物品→③解(次最优解)
- 策略3: 按价值率最大贪心, 使单位重量价值增长最快。
按价值率排序从大到小选择物品→④解(最优)

● 贪心选择的最优性证明

➤ 定理：如果 $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$ ，则 GreedyKnapsack 算法对于给定的背包问题实例生成一个最优解

➤ 证明基本思想：

把贪心解与任一最优解相比较，如果这两个解不同，就去找开始不同的第一个 x_i ，然后设法用贪心解的 x_i 去代换最优解的 x_i ，并证明最优解在分量代换之后其总价值保持不变，反复进行下去，直到新产生的最优解与贪心解完全一样，从而证明了贪心解是最优解。

- 证明：设 $(x_1, \dots, x_n)$ 是贪心算法求得的解

Case1: 所有 $x_i = 1$ 。显然该解就是最优解。

Case2: 设 $X = (1, \dots, 1, x_j, 0, \dots, 0)$ $x_j \neq 1, 1 \leq j \leq n$ 。下证 X 就是最优解。设问题的最优解 $Y = (y_1, \dots, y_n)$, 则存在 k 使得 $y_k \neq x_k$ 的最小下标（否则 $Y = X$, 得证）。

首先, 可以证明 $y_k < x_k$ 。(反证: 若 $y_k > x_k$, 则 $x_k \neq 1, \therefore k \geq j$

$$\Rightarrow \sum_{i=1}^n w_i y_i \geq \sum_{i=1}^k w_i y_i > \sum_{i=1}^k w_i x_i \geq \sum_{i=1}^j w_i x_i = c \therefore Y \text{ 不是可行解, 矛盾})$$

下面改造 Y 成为新解 $Z = (z_1, \dots, z_n)$, 并使 Z 仍为最优解。将 y_k 增加到 x_k , 从 $(y_{k+1}, \dots, y_n)$ 中减同样的重量使总量仍是 c 。即,

$$z_i = x_i \quad i = 1, 2, \dots, k; \quad \text{和} \quad w_k(z_k - y_k) = \sum_{i=k+1}^n w_i(y_i - z_i)$$

$$\begin{aligned}
\therefore \sum_{i=1}^n w_i z_i &= \sum_{i=1}^{k-1} w_i z_i + w_k z_k + \sum_{i=k+1}^n w_i z_i \\
&= \sum_{i=1}^{k-1} w_i y_i + w_k z_k + \left(\sum_{i=k+1}^n w_i y_i - w_k (z_k - y_k) \right) = \sum_{i=1}^n w_i y_i = c \\
\therefore \sum_{i=1}^n v_i z_i &= \sum_{i=1}^n v_i y_i + (z_k - y_k) v_k - \sum_{i=k+1}^n (y_i - z_i) v_i \quad // \because Z \text{ 由 } Y \text{ 变得的} \\
&= \sum_{i=1}^n v_i y_i + (z_k - y_k) w_k v_k / w_k - \sum_{i=k+1}^n (y_i - z_i) w_i v_i / w_i \\
&\geq \sum_{i=1}^n v_i y_i + \left[(z_k - y_k) w_k - \sum_{i=k+1}^n (y_i - z_i) w_i \right] v_k / w_k \quad // \text{利用 } v_i / w_i \downarrow \\
&= \sum_{i=1}^n v_i y_i
\end{aligned}$$

$\therefore Z$ 也是最优解, 且 $z_i = x_i \ i=1, \dots, k$; 重复上面过程 $\Rightarrow X$ 为最优解。

- 特殊的0-1背包问题:

如果 $w_1 \leq w_2 \leq \dots \leq w_n$, $v_1 \geq v_2 \geq \dots \geq v_n$, 则 $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$, 此时可以用贪心法求最优解。

0-1-Knapsack($v[]$, $w[]$, n , c)

```
{ //输出 $x[1 \dots n]$ 
  for  $i=1$  to  $n$  do  $x[i]=0$ ;
  value = 0.0;

  {
    if (  $w[i]<c$  )
      {  $x[i] = 1$ ;  $c -= w[i]$ ; value +=  $v[i]$ ; }
    else break;
  }
  return value;
}
```

谢谢大家